

High order FD discretization of Burgers' equation in 1D with WENO

Calcolo Scientifico (Progetto 14)

Sommario

Questo documento raccoglie la teoria e le dimostrazioni alla base del progetto: la costruzione di uno schema numerico ad alta risoluzione (WENO5 + SSP-RK3) per l'equazione di Burgers non lineare, e il suo confronto con la soluzione esatta calcolata tramite il metodo delle caratteristiche e la condizione di entropia di Oleinik. Si parte dalla derivazione della condizione di salto di Rankine-Hugoniot, si mostra perché gli schemi classici basati sullo sviluppo di Taylor (Lax-Wendroff diretto) falliscono in presenza di urti, si introduce la forma conservativa come rimedio, e infine si costruisce nel dettaglio il flusso numerico WENO5.

Indice

1	Introduzione	2
2	Formazione degli urti e soluzioni deboli	3
2.1	Il metodo delle caratteristiche	3
2.2	Tempo di rottura	3
2.3	Soluzioni deboli	4
2.4	Condizioni di entropia	6
3	L'approccio di Taylor e il suo fallimento sugli urti	7
3.1	Il problema lineare: l'equazione del trasporto	7
3.2	Il problema non lineare: Burgers (approccio diretto)	8
3.3	Fallimento dell'approccio diretto di Taylor sugli urti	8
4	Forma conservativa	9
4.1	Derivazione dalla forma integrale	9
4.2	Il flusso di Lax-Wendroff in forma conservativa	10
4.3	Conservazione della massa: approccio conservativo	11
4.4	Non conservazione della massa: approccio diretto	12
4.5	Generalizzazione: il caso lineare come caso particolare	13
4.6	Perché la forma conservativa basta: cancellazione telescopica	13
5	Il metodo WENO (Weighted Essentially Non-Oscillatory)	14
5.1	Motivazione	14
5.2	Struttura del metodo: separazione tra spazio e tempo	14
5.3	Flux splitting di Lax-Friedrichs	15
5.4	I tre sotto-stencil e la derivazione di p_0	16

5.5	Indicatori di lisciatura e pesi non lineari	17
5.6	Proprietà del metodo	18
5.7	Pseudocodice	19
6	Soluzione esatta	22
6.1	Formulazione e problema del multivalore	22
6.2	Caso $t \approx 0$	22
6.3	Estensione del dominio di ricerca	22
6.4	Ricerca delle radici e selezione entropica	23
6.5	Caso particolare: rarefazione di Riemann	24

1 Introduzione

Sappiamo $u(x, 0)$ per ogni x , e da quei valori vogliamo calcolare u_j^{n+1} nel punto x_j al tempo successivo. Per farlo dobbiamo usare i valori nei punti vicini x_{j-1} e x_{j+1} al tempo precedente, perché la derivata spaziale lega i punti tra loro: un metodo che considera più punti vicini ottiene generalmente un ordine più alto (errore minore) sulle soluzioni regolari.

Il modello che studiamo è l'equazione di Burgers non viscosa,

$$u_t + f(u)_x = 0, \quad f(u) = \frac{1}{2}u^2,$$

il prototipo più semplice di legge di conservazione scalare non lineare. Nonostante la sua semplicità, l'equazione di Burgers sviluppa discontinuità (urti) in tempo finito anche a partire da dati iniziali lisci, ed è quindi il banco di prova naturale per qualunque metodo numerico pensato per leggi di conservazione più complesse (gasdinamica, traffico, ecc.).

Obiettivo del progetto. Costruire uno schema alle differenze finite di alto ordine (WENO5 in spazio, SSP-RK3 in tempo) per approssimare $u(x, t)$, confrontarlo con la soluzione esatta ottenuta con il metodo delle caratteristiche e la condizione entropica di Oleinik, e verificarne numericamente l'ordine di convergenza rispetto a schemi di ordine più basso (upwind, WENO3).

Struttura del documento. Le Sezioni 2–4 sviluppano la teoria necessaria: formazione degli urti, condizione di Rankine-Hugoniot, motivo per cui gli schemi basati su Taylor falliscono e necessità della forma conservativa. La Sezione 5 costruisce il flusso numerico WENO5. La Sezione 6 descrive come si calcola la soluzione esatta nel codice.

Nota di versione. In questa versione la formula della soluzione esatta è stata corretta: al posto della regola euristica $u(x, t) = \min\{u_0(y) : y + u_0(y)t = x\}$ — che con più urti che interagiscono non seleziona la soluzione entropica — si usa la formula di **Lax–Oleinik** (Sezione 2.4), che prende il minimo del funzionale $J(y) = G(y) + t f^*((x - y)/t)$. La correzione è motivata e verificata numericamente nelle Sezioni 2.4, 6.1, 6.4, 6.5 e dal test `test_soluzione_esatta.py`; il codice di riferimento è `codice/burgers_weno.py` (listato in Appendice).

2 Formazione degli urti e soluzioni deboli

2.1 Il metodo delle caratteristiche

Per l'equazione di Burgers in forma non conservativa,

$$u_t + uu_x = 0,$$

cerchiamo curve $x(t)$ nel piano spazio-tempo lungo le quali u resta costante. Imponendo che la derivata totale di $u(x(t), t)$ sia nulla,

$$\frac{d}{dt}u(x(t), t) = u_t + u_x \frac{dx}{dt} = 0,$$

e confrontando con l'equazione, si vede che basta scegliere $\frac{dx}{dt} = u$. Lungo tali curve u è costante, quindi anche la loro pendenza $\frac{dx}{dt} = u$ è costante ($\frac{dx}{dt} = u_0(x_0)$): le caratteristiche sono rette (basta integrare)

$$x(t) = x_0 + u_0(x_0)t,$$

dove x_0 è la posizione iniziale e $u_0(x_0)$ il valore iniziale della soluzione in quel punto. Il valore della soluzione in (x, t) è quindi $u(x, t) = u_0(x_0)$, con x_0 soluzione dell'equazione implicita

$$x_0 + u_0(x_0)t = x. \quad (2.1)$$

2.2 Tempo di rottura

Poiché la velocità di propagazione $u_0(x_0)$ dipende dal punto di partenza, due caratteristiche uscenti da $x_1 < x_2$ possono incontrarsi: ciò accade quando u_0 è decrescente tra x_1 e x_2 . Il tempo di rottura è

$$T_b = \min_{x: u'_0(x) < 0} \left(-\frac{1}{u'_0(x)} \right),$$

il primo istante in cui la soluzione, ancora univoca, sviluppa pendenza infinita. Dopo T_b l'equazione (2.1) può avere più soluzioni x_0 per lo stesso x : la soluzione classica cessa di esistere e occorre un concetto più debole di soluzione. Per una derivazione matematica della formula consideriamo due punti iniziali $x_1 < x_2$. Le loro caratteristiche sono

$$X_1(t) = x_1 + u_0(x_1)t, \quad X_2(t) = x_2 + u_0(x_2)t.$$

Imponiamo l'incrocio $X_1(t) = X_2(t)$:

$$x_1 + u_0(x_1)t = x_2 + u_0(x_2)t \implies t = \frac{x_2 - x_1}{u_0(x_1) - u_0(x_2)}.$$

Se u_0 è decrescente ($u_0(x_1) > u_0(x_2)$), allora $t > 0$. Per ottenere il tempo in cui due caratteristiche infinitesimamente vicine si incontrano, poniamo $x_2 = x_1 + \Delta x$ con $\Delta x \rightarrow 0$ e sviluppiamo $u_0(x_2) \approx u_0(x_1) + u'_0(x_1)\Delta x$:

$$t = \frac{\Delta x}{u_0(x_1) - [u_0(x_1) + u'_0(x_1)\Delta x]} = \frac{\Delta x}{-u'_0(x_1)\Delta x} = -\frac{1}{u'_0(x_1)}.$$

Il **tempo di rottura** T_b è il primo istante in cui due caratteristiche si incontrano, ovvero il minimo di questi tempi tra tutti i punti in cui $u'_0(x) < 0$:

$$T_b = \min_{x: u'_0(x) < 0} \left(-\frac{1}{u'_0(x)} \right).$$

In parole povere il tempo di rottura è l'istante in cui un'onda (o un profilo di densità) diventa così ripida da "spezzarsi" formando un fronte d'urto.

Prima di quell'istante la curva è liscia e regolare; dopo inizia a formarsi una discontinuità, come quando un'onda del mare si accavalla su se stessa e si rompe.

2.3 Soluzioni deboli

Dopo il tempo di rottura T_b , le derivate u_t, u_x non sono definite nel punto di salto, quindi $u_t + f(u)_x = 0$ non vale più punto per punto. Tuttavia, la legge di conservazione in *forma integrale* continua ad avere senso su ogni intervallo $[x_1, x_2]$:

$$\frac{d}{dt} \int_{x_1}^{x_2} u(x, t) dx = f(u(x_1, t)) - f(u(x_2, t)). \quad (2.2)$$

Per definire rigorosamente le soluzioni che soddisfano (2.2) anche in presenza di discontinuità, si introduce il concetto di **soluzione debole**. Sia $\phi(x, t)$ una funzione test liscia a supporto compatto. Moltiplicando l'equazione per ϕ e integrando su tutto lo spazio-tempo:

$$\int_0^\infty \int_{-\infty}^{+\infty} (u_t + f(u)_x) \phi dx dt = 0.$$

Integrando per parti in t :

$$\int_0^\infty \int_{-\infty}^{+\infty} u_t \phi dx dt = - \int_{-\infty}^{+\infty} u_0(x) \phi(x, 0) dx - \int_0^\infty \int_{-\infty}^{+\infty} u \phi_t dx dt.$$

Integrando per parti in x :

$$\int_0^\infty \int_{-\infty}^{+\infty} f(u)_x \phi dx dt = - \int_0^\infty \int_{-\infty}^{+\infty} f(u) \phi_x dx dt.$$

Sommando i due contributi:

$$- \int_{-\infty}^{+\infty} u_0(x) \phi(x, 0) dx - \int_0^\infty \int_{-\infty}^{+\infty} u \phi_t dx dt - \int_0^\infty \int_{-\infty}^{+\infty} f(u) \phi_x dx dt = 0.$$

Moltiplicando per -1 :

$$\int_0^\infty \int_{-\infty}^{+\infty} (u \phi_t + f(u) \phi_x) dx dt + \int_{-\infty}^{+\infty} u_0(x) \phi(x, 0) dx = 0. \quad (2.3)$$

Questa formulazione è più debole perché non richiede che u sia derivabile: le derivate sono state trasferite sulla funzione test ϕ , che è liscia. Ogni soluzione classica (regolare) soddisfa (2.3); viceversa, una funzione che soddisfa (2.3) per ogni ϕ è una **soluzione debole**.

Condizione di Rankine-Hugoniot. Per ricavare esplicitamente la condizione di salto utilizziamo la forma integrale (2.2) equivalente alla legge di conservazione integrale (2.2) su ogni intervallo $[x_1, x_2]$. Supponiamo che l'urto sia localizzato in $x = \sigma(t)$ all'interno dell'intervallo $[x_1, x_2]$, e scomponiamo l'integrale spaziale nelle due porzioni lisce separate dalla discontinuità, applicando la regola di Leibniz per la derivazione sotto segno di integrale sugli estremi mobili. Nelle regioni in cui u è regolare vale l'equazione differenziale $u_t = -f(u)_x$; sostituendo, i termini interni si semplificano e si ottiene la condizione algebrica (2.4):

$$s = \frac{f(u^-) - f(u^+)}{u^- - u^+}, \quad s = \sigma'(t). \quad (2.4)$$

Si parte dalla legge di conservazione integrale (valida anche con urti):

$$\frac{d}{dt} \int_{x_1}^{x_2} u(x, t) dx = f(u(x_1, t)) - f(u(x_2, t)). \quad (A)$$

Sia $\Gamma : x = \sigma(t)$ la curva d'urto, con $x_1 < \sigma(t) < x_2$. Definiamo i valori della soluzione appena a sinistra e appena a destra della curva di discontinuità:

$$u^- := u(\sigma(t)^-, t) = \lim_{\varepsilon \rightarrow 0^+} u(\sigma(t) - \varepsilon, t), \quad u^+ := u(\sigma(t)^+, t) = \lim_{\varepsilon \rightarrow 0^+} u(\sigma(t) + \varepsilon, t).$$

Dove $\sigma(t)^-$ indica il limite sinistro della coordinata spaziale (cioè $x \rightarrow \sigma(t)^-$) e $\sigma(t)^+$ il limite destro ($x \rightarrow \sigma(t)^+$). Regola di Leibniz (derivata sotto integrale con estremi mobili):

$$\begin{aligned} \frac{d}{dt} \int_{x_1}^{\sigma(t)} u dx &= u(\sigma^-, t) \sigma'(t) + \int_{x_1}^{\sigma(t)} u_t dx, \\ \frac{d}{dt} \int_{\sigma(t)}^{x_2} u dx &= -u(\sigma^+, t) \sigma'(t) + \int_{\sigma(t)}^{x_2} u_t dx. \end{aligned}$$

Sostituendo in (A), con $u^- = u(\sigma^-, t)$ e $u^+ = u(\sigma^+, t)$:

$$(u^- - u^+) \sigma' + \int_{x_1}^{\sigma} u_t dx + \int_{\sigma}^{x_2} u_t dx = f(u(x_1)) - f(u(x_2)). \quad (B)$$

Nelle regioni lisce ($x \neq \sigma(t)$) vale $u_t = -f(u)_x$. Quindi:

$$\begin{aligned} \int_{x_1}^{\sigma} u_t dx &= - \int_{x_1}^{\sigma} f(u)_x dx = -f(u^-) + f(u(x_1)), \\ \int_{\sigma}^{x_2} u_t dx &= - \int_{\sigma}^{x_2} f(u)_x dx = -f(u(x_2)) + f(u^+). \end{aligned}$$

Sostituendo in (B):

$$(u^- - u^+) \sigma' - f(u^-) + f(u(x_1)) - f(u(x_2)) + f(u^+) = f(u(x_1)) - f(u(x_2)).$$

Semplificando i termini $f(u(x_1)) - f(u(x_2))$ da entrambi i lati:

$$(u^- - u^+) \sigma' - f(u^-) + f(u^+) = 0.$$

Da cui si ricava la condizione di Rankine–Hugoniot:

$$\sigma'(t) = \frac{f(u^-) - f(u^+)}{u^- - u^+} \quad \text{oppure} \quad s = \frac{\Delta f}{\Delta u}.$$

Per l'equazione di Burgers, $f(u) = \frac{1}{2}u^2$, quindi

$$s = \frac{\frac{1}{2}(u^-)^2 - \frac{1}{2}(u^+)^2}{u^- - u^+} = \frac{u^- + u^+}{2}.$$

2.4 Condizioni di entropia

Per $u^- > u^+$ la soluzione debole corrisponde a un urto fisico; per $u^- < u^+$ esiste anche un “urto di espansione” non fisico. Per selezionare la soluzione corretta si impone la **condizione di Lax**:

$$f'(u^-) > s > f'(u^+),$$

che per Burgers equivale a $u^- > u^+$.

L'urto deve saltare da un valore alto a sinistra a uno basso a destra

Una formulazione equivalente è la **condizione di Oleinik**: per ogni $x_2 > x_1$ e $t > 0$

$$u(x_2, t) - u(x_1, t) \leq \frac{x_2 - x_1}{t},$$

cioè $u(\cdot, t)$ non può crescere più rapidamente di $1/t$: è la forma integrata del requisito che escluda gli urti di rarefazione ($u^- < u^+$). Per flusso convesso ($f'' > 0$, come per Burgers) la soluzione entropica si scrive in forma chiusa con la **formula di Lax–Oleinik** (o *Hopf–Lax–Oleinik*):

$$u(x, t) = (f^*)' \left(\frac{x - y^*}{t} \right), \quad y^* = \arg \min_{y \in \mathbb{R}} \left[G(y) + t f^* \left(\frac{x - y}{t} \right) \right], \quad G(y) = \int_{y_0}^y u_0(s) ds,$$

dove $f^*(p) = \sup_u [pu - f(u)]$ è la trasformata di Legendre–Fenchel di f . Per Burgers $f^*(p) = p^2/2$, quindi $(f^*)'(p) = p$ e la formula diventa

$$u(x, t) = \frac{x - y^*}{t}, \quad y^* = \arg \min_{y \in \mathbb{R}} \left[G(y) + \frac{(x - y)^2}{2t} \right].$$

Attenzione (questo è il punto delicato, corretto in questa versione del documento). I punti stazionari del funzionale $J(y) = G(y) + t f^*((x - y)/t)$ sono esattamente le radici di $y + u_0(y)t = x$, cioè le caratteristiche che arrivano in (x, t) : restringersi a quelle radici è corretto. Quello che *non* è corretto è scegliere fra i candidati quello con $u_0(y)$ minimo, come si fa talvolta in modo euristico: la selezione entropica consiste nel prendere il candidato che **minimizza il funzionale J** . I due criteri coincidono quando la soluzione è a un solo valore o quando c'è un solo urto isolato, ma *divergono* appena più urti interagiscono: in quel caso $\min\{u_0(y) : y + u_0(y)t = x\}$ può pescare una caratteristica molto lontana (con u_0 piccolo) e restituire un valore che non è la soluzione fisica, mentre J confronta le aree $\int u_0$ pesate dal termine di Legendre–Fenchel. Si veda la Sezione 6 per la verifica numerica e il test automatico allegato al codice.

3 L'approccio di Taylor e il suo fallimento sugli urti

3.1 Il problema lineare: l'equazione del trasporto

Consideriamo l'equazione del trasporto lineare, con velocità costante $a \in \mathbb{R}$: $u_t = -au_x$. Sviluppando in serie di Taylor rispetto al tempo fino al secondo ordine,

$$u_j^{n+1} = u_j^n + \Delta t (u_t)_j^n + \frac{\Delta t^2}{2} (u_{tt})_j^n + O(\Delta t^3).$$

Il primo termine è $(u_t)_j^n = -a(u_x)_j^n$. Per il secondo, deriviamo u_t rispetto al tempo e usiamo il teorema di Schwarz per scambiare $u_{xt} = (u_t)_x$:

$$u_{tt} = \frac{\partial}{\partial t}(-au_x) = -a(u_t)_x = -a(-au_x)_x = a^2 u_{xx}.$$

Sostituendo, si ottiene la forma differenziale continua del secondo ordine:

$$u_j^{n+1} = u_j^n - \Delta t a (u_x)_j^n + \frac{\Delta t^2}{2} a^2 (u_{xx})_j^n.$$

Discretizzando le derivate spaziali con differenze centrate,

$$(u_x)_j^n \approx \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x}, \quad (u_{xx})_j^n \approx \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2},$$

si ottiene lo schema esplicito di Lax-Wendroff per il singolo nodo j :

$$u_j^{n+1} = u_j^n - \frac{a\Delta t}{2\Delta x} (u_{j+1}^n - u_{j-1}^n) + \frac{a^2\Delta t^2}{2\Delta x^2} (u_{j+1}^n - 2u_j^n + u_{j-1}^n).$$

Questo schema è l'estensione naturale del metodo di Taylor al secondo ordine per il caso lineare, ed è esatto fino a $O(\Delta t^3)$ perché la soluzione $u(x, t) = u_0(x - at)$ resta liscia quanto lo è il dato iniziale.

3.2 Il problema non lineare: Burgers (approccio diretto)

Per Burgers, in forma non conservativa $u_t = -uu_x$. Applichiamo lo stesso procedimento di Taylor. Il termine del primo ordine è $(u_t)_j^n = -u_j^n (u_x)_j^n$. Per il secondo ordine deriviamo ancora rispetto al tempo:

$$u_{tt} = \frac{\partial}{\partial t}(-uu_x) = -u_t u_x - uu_{xt}.$$

Sfruttando Schwarz ($u_{xt} = (u_t)_x$) e sostituendo $u_t = -uu_x$ e $(u_t)_x = (-uu_x)_x = -(u_x)^2 - uu_{xx}$, si ottiene:

$$u_{tt} = -(-uu_x)u_x - u(-(u_x)^2 - uu_{xx}) = u(u_x)^2 + u(u_x)^2 + u^2 u_{xx} = 2u(u_x)^2 + u^2 u_{xx}.$$

Sostituendo u_t e u_{tt} nello sviluppo di Taylor:

$$u_j^{n+1} = u_j^n - \Delta t (u_j^n u_x)_j^n + \frac{\Delta t^2}{2} (2u_j^n (u_x)_j^n + (u_j^n)^2 u_{xx})_j^n.$$

Discretizzando $(u_x)_j^n \approx \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x}$ e $(u_{xx})_j^n \approx \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2}$, si ottiene lo schema esplicito

$$u_j^{n+1} = u_j^n - \frac{\Delta t}{2\Delta x} u_j^n (u_{j+1}^n - u_{j-1}^n) + \frac{\Delta t^2}{2} \left[2u_j^n \left(\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \right)^2 + (u_j^n)^2 \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2} \right].$$

Questo schema è la traduzione letterale del “trucco di Lax-Wendroff” al caso non lineare: approssima direttamente il valore puntuale u_j^{n+1} tramite Taylor, senza passare per la forma integrale.

3.3 Fallimento dell’approccio diretto di Taylor sugli urti

Lo sviluppo di Taylor richiede la regolarità di u , ma in un urto u_x diverge e le derivate superiori non esistono. Lo schema genera oscillazioni di Gibbs, perde consistenza (l’errore di troncamento contiene u_{xxx} , infinito all’urto) e viola i vincoli fisici. Il problema più grave è la non conservazione della massa discreta, che produce una velocità dell’urto $s_{num} \neq s$ errata, anche al raffinare la griglia. Il rimedio è sostituire l’approssimazione puntuale con quella della forma integrale, valida anche attraverso le discontinuità.

4 Forma conservativa

4.1 Derivazione dalla forma integrale

Prendiamo una cella spaziale $[x_{j-1/2}, x_{j+1/2}]$ e integriamo $u_t + f(u)_x = 0$ su questa cella e sull'intervallo temporale $[t_n, t_{n+1}]$:

$$\int_{t_n}^{t_{n+1}} \int_{x_{j-1/2}}^{x_{j+1/2}} (u_t + f(u)_x) dx dt = 0.$$

Scambiando l'ordine di integrazione:

$$\int_{x_{j-1/2}}^{x_{j+1/2}} \left(\int_{t_n}^{t_{n+1}} u_t dt \right) dx + \int_{t_n}^{t_{n+1}} \left(\int_{x_{j-1/2}}^{x_{j+1/2}} f(u)_x dx \right) dt = 0.$$

Il primo integrale in t è $u(x, t_{n+1}) - u(x, t_n)$; il secondo in x è $f(u(x_{j+1/2}, t)) - f(u(x_{j-1/2}, t))$ per il teorema fondamentale del calcolo. Dividendo per Δx e definendo la media di cella

$$U_j^n = \frac{1}{\Delta x} \int_{x_{j-1/2}}^{x_{j+1/2}} u(x, t_n) dx,$$

si ottiene

$$U_j^{n+1} = U_j^n - \frac{1}{\Delta x} \left(\int_{t_n}^{t_{n+1}} f(u(x_{j+1/2}, t)) dt - \int_{t_n}^{t_{n+1}} f(u(x_{j-1/2}, t)) dt \right).$$

La quantità $\frac{1}{\Delta t} \int_{t_n}^{t_{n+1}} f(u(x_{j+1/2}, t)) dt$ è il **flusso medio** attraverso il bordo destro della cella; la chiamiamo $F_{j+1/2}^n$. Con la regola del punto medio (accurata al second'ordine),

$$F_{j+1/2}^n \approx f(u(x_{j+1/2}, t_{n+1/2})) + O(\Delta t^2).$$

4.2 Il flusso di Lax-Wendroff in forma conservativa

Sviluppiamo in Taylor *in tempo* la funzione $g(x, t) = f(u(x, t))$ attorno a t_n , fermandoci al prim'ordine (l'errore sarà $O(\Delta t^2)$ dopo aver moltiplicato per Δt):

$$g(x_{j+1/2}, t_{n+1/2}) = g(x_{j+1/2}, t_n) + \frac{\Delta t}{2} g_t(x_{j+1/2}, t_n) + O(\Delta t^2).$$

Per la regola della catena $g_t = f'(u)u_t$; usando $u_t = -f(u)_x = -g_x$ e ponendo $a(u) = f'(u)$, si ha $g_t = -a(u)g_x$. Quindi

$$F_{j+1/2}^n = g(x_{j+1/2}, t_n) - \frac{\Delta t}{2} a(u) g_x(x_{j+1/2}, t_n) + O(\Delta t^2).$$

Approssimando i termini al bordo con i valori nelle celle j e $j+1$: la media aritmetica per g , il rapporto incrementale centrato per g_x , e la media dei due stati per $a(u)$,

$$g(x_{j+1/2}, t_n) \approx \frac{g_j + g_{j+1}}{2}, \quad g_x(x_{j+1/2}, t_n) \approx \frac{g_{j+1} - g_j}{\Delta x}, \quad a_{j+1/2} = \frac{U_j^n + U_{j+1}^n}{2},$$

si arriva alla **formula del flusso di Lax-Wendroff in forma conservativa**:

$$F_{j+1/2}^n = \frac{g_j + g_{j+1}}{2} - \frac{\Delta t}{2\Delta x} a_{j+1/2} (g_{j+1} - g_j).$$

La differenza sostanziale rispetto all'approccio diretto della Sezione 3 è che qui si approssima il *flusso integrato al bordo* $F_{j+1/2}^n = f(u(x_{j+1/2}, t_{n+1/2}))$, quantità che esiste sempre perché deriva dalla forma integrale, mentre l'approccio diretto approssima la soluzione puntuale u_j^{n+1} , che sull'urto non esiste in senso classico.

4.3 Conservazione della massa: approccio conservativo

Consideriamo lo schema $u_j^{n+1} = u_j^n - \frac{\Delta t}{\Delta x}(F_{j+1/2}^n - F_{j-1/2}^n)$ su un dominio periodico con J celle, e definiamo la massa discreta $\mathcal{M}^n = \Delta x \sum_{j=1}^J u_j^n$. Allora

$$\mathcal{M}^{n+1} = \Delta x \sum_{j=1}^J u_j^n - \Delta t \sum_{j=1}^J (F_{j+1/2}^n - F_{j-1/2}^n) = \mathcal{M}^n - \Delta t \sum_{j=1}^J (F_{j+1/2}^n - F_{j-1/2}^n).$$

La somma dei flussi è telescopica:

$$\sum_{j=1}^J (F_{j+1/2}^n - F_{j-1/2}^n) = F_{J+1/2}^n - F_{1-1/2}^n,$$

e per periodicità $F_{J+1/2}^n = F_{1-1/2}^n$, quindi la somma è nulla e $\mathcal{M}^{n+1} = \mathcal{M}^n$. Poiché ciò vale per ogni n , $\mathcal{M}^n = \mathcal{M}^0$ per ogni n : la forma conservativa preserva *esattamente* il bilancio di massa discreto, indipendentemente da quanto sia ripida la soluzione. Per il teorema di Lax-Wendroff (Sezione 4.5), se uno schema conservativo e consistente converge, converge a una soluzione debole corretta: la velocità dell'urto numerico coincide con quella di Rankine-Hugoniot.

4.4 Non conservazione della massa: approccio diretto

Consideriamo ora lo schema ottenuto in Sezione 3.2 dallo sviluppo di Taylor diretto sulla forma non conservativa, discretizzando le derivate spaziali con differenze centrate, si ottiene:

$$u_j^{n+1} = u_j^n - \frac{\Delta t}{2\Delta x} u_j (u_{j+1} - u_{j-1}) + \frac{\Delta t^2}{2\Delta x^2} u_j^2 (u_{j+1} - 2u_j + u_{j-1}) + \frac{\Delta t^2}{4\Delta x^2} u_j (u_{j+1} - u_{j-1})^2.$$

Calcoliamo ora la massa discreta $\mathcal{M}^n = \Delta x \sum_{j=1}^J u_j^n$. La variazione di massa dal passo n al passo $n+1$ è:

$$\mathcal{M}^{n+1} - \mathcal{M}^n = -\frac{\Delta t}{2} \sum_{j=1}^J u_j (u_{j+1} - u_{j-1}) + \frac{\Delta t^2}{2\Delta x} \sum_{j=1}^J u_j^2 (u_{j+1} - 2u_j + u_{j-1}) + \frac{\Delta t^2}{4\Delta x} \sum_{j=1}^J u_j (u_{j+1} - u_{j-1})^2.$$

Per periodicità, il termine $O(\Delta t)$ si annulla (è una traslazione di indici):

$$\sum_{j=1}^J u_j^n u_{j+1}^n - \sum_{j=1}^J u_j^n u_{j-1}^n = 0.$$

Per i termini $O(\Delta t^2)$, definiamo la quantità

$$\mathcal{R}(u^n) = \sum_{j=1}^J u_j^2 (u_{j+1} - 2u_j + u_{j-1}) + \frac{1}{2} \sum_{j=1}^J u_j (u_{j+1} - u_{j-1})^2.$$

Espandendo il primo termine e traslando gli indici ($\sum_j u_j^2 u_{j-1} = \sum_j u_{j+1}^2 u_j$), si ha:

$$\sum_{j=1}^J u_j^2 (u_{j+1} - 2u_j + u_{j-1}) = \sum_{j=1}^J \left[u_j u_{j+1} (u_j + u_{j+1}) - 2u_j^3 \right].$$

Il secondo termine non può essere riscritto come una differenza telescopica e non si semplifica con il primo in modo da dare zero identicamente. In generale,

$$\mathcal{R}(u^n) \neq 0,$$

come si verifica immediatamente anche su profili semplici non costanti (ad esempio, su una griglia periodica con valori $u = [1, 2, 3]$, $\mathcal{R} \neq 0$).

Pertanto, la variazione di massa è

$$\mathcal{M}^{n+1} = \mathcal{M}^n + \frac{\Delta t^2}{2\Delta x} \mathcal{R}(u^n) \neq \mathcal{M}^n.$$

In presenza di un urto, le differenze $u_{j+1}^n - u_j^n$ sono $O(1)$ indipendentemente dal raffinamento della griglia, quindi $\mathcal{R}(u^n) = O(1)$ non tende a zero. Sotto la condizione CFL ($\lambda = \Delta t / \Delta x$ costante), l'errore di massa per singolo passo temporale scala come

$$\frac{\Delta t^2}{2\Delta x} \mathcal{R}(u^n) = \frac{\Delta t}{2} \lambda \mathcal{R}(u^n) = O(\Delta t).$$

Poiché per raggiungere un tempo finale T sono necessari $N_t = O(1/\Delta t)$ passi temporali, l'errore di massa accumulato è $O(1)$: si tratta di un deficit permanente, che non scompare al raffinare la griglia. Questo viola la condizione di Rankine-Hugoniot a livello discreto e costringe l'urto numerico a propagarsi a una velocità $s_{\text{num}} \neq s$ errata, anche nel limite di griglia fine.

4.5 Generalizzazione: il caso lineare come caso particolare

Verifichiamo che la formula del flusso conservativo generalizza correttamente il metodo di Lax-Wendroff lineare della Sezione 3.1. Per $f(u) = au$ con a costante si ha $g_j = au_j$ e $a(u) = f'(u) = a$. Sostituendo nella formula del flusso,

$$F_{j+1/2}^n = \frac{g_j + g_{j+1}}{2} - \frac{\Delta t}{2\Delta x} a_{j+1/2} (g_{j+1} - g_j),$$

si ottiene

$$F_{j+1/2} = \frac{a}{2} (u_j + u_{j+1}) - \frac{a^2 \Delta t}{2\Delta x} (u_{j+1} - u_j).$$

Calcolando $F_{j+1/2} - F_{j-1/2}$ e sostituendo in $U_j^{n+1} = U_j^n - \frac{\Delta t}{\Delta x} (F_{j+1/2} - F_{j-1/2})$ si ritrova esattamente lo schema di Lax-Wendroff lineare:

$$U_j^{n+1} = U_j^n - \frac{a\Delta t}{2\Delta x} (U_{j+1} - U_{j-1}) + \frac{a^2 \Delta t^2}{2\Delta x^2} (U_{j+1} - 2U_j + U_{j-1}).$$

Per $f(u)$ lineare, dunque, l'approccio diretto e quello conservativo coincidono (grazie alla linearità): è per questo che nella Sezione 3.1 il metodo di Taylor diretto è ineccepibile. Solo quando $f(u)$ è non lineare le due strade divergono, e solo la forma conservativa garantisce la corretta velocità dell'urto.

4.6 Perché la forma conservativa basta: cancellazione telescopica

Il punto cruciale, che riassume l'intera sezione, è puramente algebrico. Scrivendo l'aggiornamento come $U_j^{n+1} = U_j^n - \frac{\Delta t}{\Delta x} (F_{j+1/2} - F_{j-1/2})$ e sommando su tutte le celle (bordo periodico o flussi nulli), tutti i flussi interni si cancellano a coppie:

$$\sum_j U_j^{n+1} \Delta x = \sum_j U_j^n \Delta x.$$

La quantità totale si conserva *esattamente*, qualunque sia la ripidità della soluzione: è questa proprietà, e non l'accuratezza puntuale, a garantire la corretta velocità dell'urto secondo Rankine-Hugoniot. Questo principio – forma conservativa $\Rightarrow$ massa esatta $\Rightarrow$ urto corretto – è lo stesso che useremo per costruire il metodo WENO nella prossima sezione: cambierà soltanto *come* si approssima il flusso $F_{j+1/2}$, non la struttura conservativa dello schema.

5 Il metodo WENO (Weighted Essentially Non-Oscillatory)

5.1 Motivazione

Gli schemi conservativi del secondo ordine come Lax-Wendroff (Sezione 4) garantiscono la corretta velocità dell'urto, ma restano sviluppi di Taylor *applicati al flusso*: in prossimità di una discontinuità producono comunque oscillazioni di Gibbs, perché il polinomio ricostruito tenta di interpolare punti da entrambi i lati del salto. Gli schemi del primo ordine come Godunov sono monotoni (non oscillano) ma eccessivamente dissipativi: l'urto viene spalmato su molte celle.

L'idea dei metodi ENO (Essentially Non-Oscillatory) è: se uno stencil attraversa una discontinuità, non usarlo per la ricostruzione. Per ogni bordo di cella si valutano più stencil candidati e si sceglie quello più liscio. I metodi WENO generalizzano l'idea usando una *media pesata* di tutti gli stencil, privilegiando quelli lisci: nelle zone regolari si recupera così l'ordine massimo possibile, mentre vicino agli urti i pesi degli stencil "cattivi" si annullano automaticamente.

5.2 Struttura del metodo: separazione tra spazio e tempo

Il metodo si articola in due fasi indipendenti.

Fase 1 (WENO, spazio): si sostituisce $f(u)_x$ con un operatore numerico $L(U)_j$, ottenendo per ogni cella un'equazione differenziale ordinaria nel tempo,

$$\frac{du_j}{dt} = L(U(t))_j = \underbrace{-\frac{1}{\Delta x}(\hat{f}_{j+1/2}(t) - \hat{f}_{j-1/2}(t))}_{\text{Stima numerica di } -f(u)_x \text{ in } x=x_j}$$

dove $U(t) = (u_1(t), \dots, u_N(t))^T$ è il vettore delle incognite all'istante t . Si noti che L **non dipende esplicitamente** da t (nella formula non compare mai la variabile t); la dipendenza dal tempo è **solo implicita**, perché L viene valutata sui valori $U(t)$ che cambiano nel tempo. Il problema diventa quindi un sistema di N ODE accoppiate (problema di Cauchy):

$$\frac{dU}{dt} = L(U(t)), \quad U(0) = U_0,$$

Fase 2 (Runge-Kutta, tempo): l'integrale esatto $U(t_{n+1}) = U(t_n) + \int_{t_n}^{t_{n+1}} L(U(\tau)) d\tau$ va approssimato con una quadratura temporale di ordine coerente con quello spaziale. Eulero esplicito darebbe solo ordine 1, sprecando l'accuratezza di WENO5; si usa quindi il metodo **SSP-RK3** (Strong Stability Preserving Runge-Kutta a 3 stadi). Ponendo $U^n = U(t_n)$, e indicando con $U^{(1)} \approx U(t_n + \Delta t)$ e $U^{(2)} \approx U(t_n + \frac{1}{2}\Delta t)$ le soluzioni approssimate ai tempi intermedi, gli stadi sono:

$$\begin{aligned} U^{(1)} &= U^n + \Delta t L(U^n), \\ U^{(2)} &= \frac{3}{4}U^n + \frac{1}{4}(U^{(1)} + \Delta t L(U^{(1)})), \\ U^{n+1} &= \frac{1}{3}U^n + \frac{2}{3}(U^{(2)} + \Delta t L(U^{(2)})). \end{aligned}$$

Poiché $\Delta t \propto \Delta x$, l'errore temporale scala come Δx^3 , mentre quello spaziale come Δx^5 . Per Δx sufficientemente piccolo, il termine Δx^3 domina, quindi l'ordine globale del metodo è 3.

5.3 Flux splitting di Lax-Friedrichs

Al tempo t_n , noto il vettore $U^n = U(t_n)$, si calcolano i flussi fisici puntuali $f_j(t_n) = \frac{1}{2}(U_j(t_n))^2$. Si applica lo split di Lax-Friedrichs per separare l'informazione che viaggia verso destra da quella che viaggia verso sinistra:

$$f^+(u) = \frac{1}{2}(f(u) + \alpha u), \quad f^-(u) = \frac{1}{2}(f(u) - \alpha u), \quad \alpha = \max_j |U_j(t_n)|,$$

Questa è la scelta standard per f^+ ed f^- : garantisce che le derivate abbiano segno costante su tutto il dominio e che la loro somma restituisca esattamente il flusso originale $f(u)$. Prendiamo $\alpha = \max_j |U_j(t_n)|$ in modo che, per ogni punto della griglia, valga $-\alpha \leq u_j \leq \alpha$; di conseguenza, siccome $f'(u) = u$ per Burgers

$$(f^+)'(u_j) = \frac{1}{2}(u_j + \alpha) \geq 0, \quad (f^-)'(u_j) = \frac{1}{2}(u_j - \alpha) \leq 0 \quad \text{per ogni } j,$$

cioè le due componenti viaggiano effettivamente in direzioni opposte e ben definite. Se avessimo scelto $\alpha < \max_j |u_j|$, esisterebbe almeno un punto j in cui $u_j > \alpha$ (se il massimo è positivo) o $u_j < -\alpha$ (se il minimo è negativo); in quel punto avremmo $(f^-)'(u_j) > 0$ oppure $(f^+)'(u_j) < 0$, violando le condizioni di upwind e rendendo instabile la ricostruzione di WENO su quel bordo.

Si sceglie $\alpha = \max_j |U_j|$ (il minimo valore che garantisce la proprietà per tutti i punti; valori più grandi sono ammissibili ma introducono maggiore dissipazione numerica). Grazie a questa proprietà, f^+ si ricostruisce con uno stencil sbilanciato a sinistra (upwind per l'onda che va a destra) e f^- con uno stencil sbilanciato a destra. Il flusso numerico finale al bordo $j + \frac{1}{2}$ all'istante t_n è quindi:

$$\hat{f}_{j+1/2}(t_n) = \hat{f}_{j+1/2}^+(t_n) + \hat{f}_{j+1/2}^-(t_n).$$

Nel seguito descriviamo la ricostruzione di $\hat{f}_{j+1/2}^+(t_n)$; quella di $\hat{f}_{j+1/2}^-(t_n)$ è analoga con indici specchiati. Si ricordi che durante i sottostadi di Runge-Kutta, lo stesso procedimento verrà ripetuto valutando L sui vettori intermedi $U^{(1)}$ e $U^{(2)}$, producendo quindi le stime $\hat{f}_{j+1/2}(t_n + \Delta t)$ e $\hat{f}_{j+1/2}(t_n + \frac{1}{2}\Delta t)$ necessarie all'integrazione temporale.

5.4 I tre sotto-stencil e la derivazione di p_0

Per WENO5 i tre sotto-stencil per il bordo $x_{j+1/2}$ sono

$$S_0 = \{j-2, j-1, j\}, \quad S_1 = \{j-1, j, j+1\}, \quad S_2 = \{j, j+1, j+2\},$$

e su ciascuno si costruisce un polinomio $p_r(x)$ di grado 2, valutato poi al bordo $x_{j+1/2}$. Mostriamo la derivazione di p_0 sullo stencil S_0 . Nel metodo dei volumi finiti f_j è una media di cella, $f_j = \frac{1}{\Delta x} \int_{x_{j-1/2}}^{x_{j+1/2}} f(x) dx$; cerchiamo $p(x) = ax^2 + bx + c$ la cui media sulle tre celle di S_0 riproduca f_{j-2}, f_{j-1}, f_j . Con $\Delta x = 1, j = 0$:

$$\int_{-5/2}^{-3/2} p dx = f_{-2}, \quad \int_{-3/2}^{-1/2} p dx = f_{-1}, \quad \int_{-1/2}^{1/2} p dx = f_0.$$

Calcolando gli integrali si ottiene il sistema

$$\begin{cases} \frac{49}{12}a - 2b + c = f_{-2}, \\ \frac{13}{12}a - b + c = f_{-1}, \\ \frac{1}{12}a + c = f_0. \end{cases}$$

Sottraendo la seconda equazione dalla prima e la terza dalla seconda:

$$3a - b = f_{-2} - f_{-1}, \quad a - b = f_{-1} - f_0.$$

Sottraendo queste due tra loro: $2a = f_{-2} - 2f_{-1} + f_0$, quindi

$$a = \frac{f_{-2} - 2f_{-1} + f_0}{2}, \quad b = a - (f_{-1} - f_0) = \frac{f_{-2} - 4f_{-1} + 3f_0}{2}, \quad c = f_0 - \frac{a}{12}.$$

Valutando in $x = 1/2$ (il bordo $x_{j+1/2}$):

$$p_0\left(\frac{1}{2}\right) = \frac{a}{4} + \frac{b}{2} + c = \frac{1}{3}f_{j-2} - \frac{7}{6}f_{j-1} + \frac{11}{6}f_j.$$

Con lo stesso procedimento, traslando gli indici sugli stencil S_1 e S_2 , si ottengono gli altri due polinomi:

$$p_1 = -\frac{1}{6}f_{j-1} + \frac{5}{6}f_j + \frac{1}{3}f_{j+1}, \quad p_2 = \frac{1}{3}f_j + \frac{5}{6}f_{j+1} - \frac{1}{6}f_{j+2}.$$

Se combinati con pesi costanti $d_0 = \frac{1}{10}, d_1 = \frac{3}{5}, d_2 = \frac{3}{10}$ (i **pesi lineari ottimali**, con $d_0 + d_1 + d_2 = 1$), i tre polinomi riproducono esattamente il polinomio di quarto grado che interpola tutti e cinque i punti dello stencil globale, dando ordine 5 nelle zone lisce. Vicino a un urto, però, questi pesi userebbero anche stencil che attraversano il salto: vanno resi adattativi. Questi pesi d_0, d_1, d_2 si calcolano risolvendo il sistema $P_{j+1/2} = d_0p_0 + d_1p_1 + d_2p_2$ dove P è il polinomio che risolve

$$\begin{cases} \int_{-5/2}^{-3/2} (\alpha x^4 + \beta x^3 + \gamma x^2 + \delta x + \eta) dx = f_{-2} \\ \int_{-3/2}^{-1/2} (\alpha x^4 + \beta x^3 + \gamma x^2 + \delta x + \eta) dx = f_{-1} \\ \int_{-1/2}^{1/2} (\alpha x^4 + \beta x^3 + \gamma x^2 + \delta x + \eta) dx = f_0 \\ \int_{1/2}^{3/2} (\alpha x^4 + \beta x^3 + \gamma x^2 + \delta x + \eta) dx = f_1 \\ \int_{3/2}^{5/2} (\alpha x^4 + \beta x^3 + \gamma x^2 + \delta x + \eta) dx = f_2 \end{cases}$$

5.5 Indicatori di lisciatura e pesi non lineari

Per ogni stencil S_r si definisce un **indicatore di lisciatura** β_r , piccolo nelle zone regolari e grande se lo stencil attraversa una discontinuità:

$$\begin{aligned}\beta_0 &= \frac{13}{12}(f_{j-2} - 2f_{j-1} + f_j)^2 + \frac{1}{4}(f_{j-2} - 4f_{j-1} + 3f_j)^2, \\ \beta_1 &= \frac{13}{12}(f_{j-1} - 2f_j + f_{j+1})^2 + \frac{1}{4}(f_{j-1} - f_{j+1})^2, \\ \beta_2 &= \frac{13}{12}(f_j - 2f_{j+1} + f_{j+2})^2 + \frac{1}{4}(3f_j - 4f_{j+1} + f_{j+2})^2,\end{aligned}$$

dove il primo addendo misura la curvatura (derivata seconda discreta) e il secondo la pendenza (derivata prima discreta) all'interno dello stencil. Mostriamo sotto la derivazione di β_0 :

$$p_0(x) = ax^2 + bx + c, \quad a = \frac{f_{-2} - 2f_{-1} + f_0}{2}, \quad b = \frac{f_{-2} - 4f_{-1} + 3f_0}{2}$$

$$p'_0(x) = 2ax + b, \quad p''_0(x) = 2a$$

$$\beta_0 = \underbrace{\int_{-1/2}^{1/2} (p'_0)^2 dx}_{\frac{a^2}{3} + b^2} + \underbrace{\int_{-1/2}^{1/2} (p''_0)^2 dx}_{4a^2}$$

$$\frac{a^2}{3} + b^2 + 4a^2 = \frac{a^2}{3} + 4a^2 + b^2 = \frac{13a^2}{3} + b^2$$

$$\frac{13a^2}{3} = \frac{13}{3} \left(\frac{f_{-2} - 2f_{-1} + f_0}{2} \right)^2 = \frac{13}{12} (f_{-2} - 2f_{-1} + f_0)^2$$

$$b^2 = \frac{1}{4} (f_{-2} - 4f_{-1} + 3f_0)^2$$

$$\beta_0 = \frac{13}{12} (f_{-2} - 2f_{-1} + f_0)^2 + \frac{1}{4} (f_{-2} - 4f_{-1} + 3f_0)^2$$

I pesi non lineari si costruiscono da d_r e β_r :

$$\alpha_r = \frac{d_r}{(\beta_r + \varepsilon)^2}, \quad \omega_r = \frac{\alpha_r}{\alpha_0 + \alpha_1 + \alpha_2}, \quad \varepsilon = 10^{-6} \text{ (evita la divisione per zero).}$$

Se β_r è piccolo (zona liscia), $\omega_r \approx d_r$ e si recupera il quinto ordine; se β_r è grande (stencil che attraversa l'urto), $\omega_r \approx 0$ e quello stencil viene escluso dalla ricostruzione. I pesi non si annullano mai esattamente (da qui 'Essentially' di WENO), ma diventano trascurabili. Il flusso ricostruito al bordo è infine

$$\hat{f}_{j+1/2}^+ = \omega_0 p_0 + \omega_1 p_1 + \omega_2 p_2.$$

5.6 Proprietà del metodo

Il metodo WENO5 gode di quattro proprietà fondamentali. La conservatività è garantita dalla struttura stessa dello schema, $U_j^{n+1} = U_j^n - \frac{\Delta t}{\Delta x}(\hat{f}_{j+1/2} - \hat{f}_{j-1/2})$, per cui la cancellazione telescopica dei flussi ai bordi interni assicura la conservazione esatta della massa totale su dominio periodico (Sezione 4.6). L'assenza di oscillazioni di Gibbs deriva dal comportamento dei pesi non lineari: in prossimità di un urto, gli stencil che lo attraversano ricevono $\omega_r \approx 0$, escludendo di fatto dalla ricostruzione i dati provenienti dal lato opposto della discontinuità. Nelle zone lisce, dove gli indicatori β_r sono confrontabili, i pesi non lineari tendono ai pesi ottimali $\omega_r \approx d_r$, e la combinazione dei tre polinomi recupera l'accuratezza di quinto ordine. Infine, la selezione della soluzione entropica è garantita dalla dissipazione numerica introdotta dallo split di Lax-Friedrichs, che orienta la ricostruzione nella direzione delle caratteristiche e seleziona la soluzione debole entropica coerente con la condizione di Oleinik (Sezione 2.4).

5.7 Pseudocodice

Lo schema completo si articola nei seguenti passaggi, ripetuti per ogni passo temporale $t_n \rightarrow t_{n+1}$ con l'integratore SSP-RK3.

1. Calcolo dell'operatore spaziale $L(U)$.

Dato un vettore $U = (U_1, \dots, U_N)$ rappresentante la soluzione in tutte le celle in un dato istante, si esegue:

(a) **Flussi fisici:** per ogni cella $j = 0, \dots, N - 1$:

$$f_j = \frac{1}{2} U_j^2.$$

(b) **Costante di Lax-Friedrichs:**

$$\alpha = \max_j |U_j|.$$

(c) **Flux splitting:** per ogni cella $j = 0, \dots, N - 1$:

$$f_j^+ = \frac{1}{2} (f_j + \alpha U_j), \quad f_j^- = \frac{1}{2} (f_j - \alpha U_j).$$

(d) **Calcolo dei flussi numerici su tutti i bordi:**

Per ogni bordo $k = j + \frac{1}{2}$ con $j = 0, \dots, N - 1$:

- Si ricostruisce $\hat{f}_k^+$ applicando la procedura WENO5 allo stencil sinistro (left-biased) sui valori:

$$f_{j-2}^+, f_{j-1}^+, f_j^+, f_{j+1}^+, f_{j+2}^+.$$

- Si ricostruisce $\hat{f}_k^-$ applicando la procedura WENO5 allo stencil destro (right-biased) sui valori:

$$f_{j-1}^-, f_j^-, f_{j+1}^-, f_{j+2}^-, f_{j+3}^-.$$

- Il flusso numerico al bordo è la somma:

$$F_k = \hat{f}_k^+ + \hat{f}_k^-.$$

Valori con indici fuori dal dominio sono ottenuti dalle condizioni al contorno. Poiché f^+ viaggia sempre a destra (ha derivata positiva), l'informazione per f^+ arriva da sinistra. Quindi per ricostruire f^+ al bordo $j + \frac{1}{2}$ si usa uno stencil sbilanciato a sinistra. Analogamente, poiché f^- viaggia sempre a sinistra (ha derivata negativa), l'informazione per f^- arriva da destra; per ricostruire f^- al bordo $j + \frac{1}{2}$ si usa quindi uno stencil sbilanciato a destra.

(e) **Calcolo di $L(U)$:** per ogni cella $j = 0, \dots, N - 1$:

$$L(U)_j = -\frac{1}{\Delta x} (F_{j+1/2} - F_{j-1/2}).$$

2. Integrazione temporale SSP-RK3.

Dato lo stato corrente U^n al tempo t_n , si eseguono i tre stadi:

$$\begin{aligned} U^{(1)} &= U^n + \Delta t L(U^n), \\ U^{(2)} &= \frac{3}{4}U^n + \frac{1}{4}U^{(1)} + \frac{1}{4}\Delta t L(U^{(1)}), \\ U^{n+1} &= \frac{1}{3}U^n + \frac{2}{3}U^{(2)} + \frac{2}{3}\Delta t L(U^{(2)}). \end{aligned}$$

Ogni chiamata di $L(\cdot)$ ricalcola l'intera procedura al punto 1 sui valori correnti.

3. Condizioni al contorno.

Dopo ogni aggiornamento (sia nei sottostadi di Runge-Kutta sia al termine del passo temporale), si applicano le condizioni al contorno periodiche:

$$U_{j+J} = U_j, \quad \text{per ogni } j,$$

dove J è il numero totale di celle del dominio.

Il passo temporale Δt viene adattato dinamicamente secondo la condizione CFL:

$$\Delta t = \text{CFL} \cdot \frac{\Delta x}{\max_j |U_j^n|},$$

con $\text{CFL} \leq 0.5$ per garantire la stabilità di SSP-RK3 con lo splitting di Lax-Friedrichs.

Il metodo WENO5 con SSP-RK3 combina i pregi degli schemi precedenti: la conservatività esatta di Lax-Wendroff (che garantisce la corretta velocità dell'urto), l'assenza di oscillazioni tipica degli schemi monotoni come Godunov, e l'elevata accuratezza spaziale (quinto ordine) nelle regioni lisce, pur mantenendo un profilo d'urto estremamente localizzato su poche celle.

Riporto sotto una breve dimostrazione della necessità della condizione CFL:

La curva caratteristica $X(t)$ soddisfa $\frac{d}{dt}u(X(t), t) = 0$. Sviluppando: $u_t + u_x X' = 0$. Confrontando con $u_t + uu_x = 0$ si ha $X' = u$. Integrando $dX = u dt$ tra t_0 e t : $\int_{X(t_0)}^{X(t)} dX = \int_{t_0}^t u(X(s), s) ds$. Poiché u è costante lungo la caratteristica, $u(X(s), s) = u(X(t_0), t_0)$, quindi $X(t) = X(t_0) + u(X(t_0), t_0)(t - t_0)$.

Consideriamo il nodo (X_j, t_{n+1}) . Scegliamo $t_0 = t_{n+1}$ e $X(t_{n+1}) = X_j$. La caratteristica è $X(t) = X_j + u(X_j, t_{n+1})(t - t_{n+1})$. Valutandola in $t = t_n$: $X(t_n) = X_j + u(X_j, t_{n+1})(t_n - t_{n+1})$. Poniamo $x^* := X(t_n)$. Poiché u è costante lungo $X(t)$, si ha $u(X(t_n), t_n) = u(X(t_{n+1}), t_{n+1})$, cioè $u(x^*, t_n) = u(X_j, t_{n+1})$. Sostituendo nella precedente e definendo $\Delta t = t_{n+1} - t_n$: $x^* = X_j - u(x^*, t_n) \Delta t$. Quindi il valore $u(X_j, t_{n+1})$ dipende esclusivamente da $u(x^*, t_n)$. Questo è il dominio di dipendenza fisico: un singolo punto x^* al tempo t_n .

Ora passiamo allo schema numerico. Per calcolare u_j^{n+1} , uno schema esplicito alle differenze finite usa un dominio di dipendenza numerico, cioè un insieme di nodi al tempo t_n . Nel caso minimale a 3 punti (upwind o centrato), lo stencil è l'intervallo $[X_{j-1}, X_{j+1}] = [X_j - \Delta x, X_j + \Delta x]$. Affinché lo schema possa convergere alla soluzione esatta, il dominio fisico deve essere contenuto in quello numerico, altrimenti lo schema ignorerebbe l'informazione da cui la soluzione dipende realmente. Deve quindi valere $X_j - \Delta x \leq x^* \leq X_j + \Delta x$. Sostituendo $x^* = X_j - u(x^*, t_n) \Delta t$: $-\Delta x \leq -u(x^*, t_n) \Delta t \leq \Delta x$,

che equivale a $|u(x^*, t_n)|\Delta t \leq \Delta x$. Poiché la velocità u varia nello spazio e la disuguaglianza deve valere per ogni cella j , si prende il caso peggiore sull'intero dominio: $\max_j |u_j^n| \cdot \frac{\Delta t}{\Delta x} \leq 1$. Da cui il vincolo $\Delta t \leq \frac{\Delta x}{\max_j |u_j^n|}$. Nella pratica si introduce un coefficiente di sicurezza $\text{CFL} \leq 0.5$ (dovuto alla stabilità effettiva di WENO5+SSP-RK3), ottenendo $\Delta t = \text{CFL} \cdot \frac{\Delta x}{\max_j |u_j^n|}$. Questa condizione è necessaria: se violata, la caratteristica fisica esce dallo stencil numerico, e lo schema non può convergere alla soluzione debole corretta.

6 Soluzione esatta

6.1 Formulazione e problema del multivalore

Come visto in Sezione 2.1, $u(x, t) = u_0(x_0)$ con x_0 soluzione di $x_0 + u_0(x_0)t = x$. Dopo il tempo di rottura T_b questa equazione può avere più soluzioni x_0 per lo stesso x : le caratteristiche si incrociano e la soluzione “a più valori” non ha senso fisico. Fra queste radici bisogna scegliere quella che realizza il minimo del funzionale di Lax–Oleinik (Sezione 2.4):

$$J(y) = G(y) + \frac{(x - y)^2}{2t}, \quad u(x, t) = \frac{x - y^*}{t}, \quad y^* = \arg \min_y J(y),$$

dove $G(y) = \int^y u_0(s) ds$ è una primitiva di u_0 e y varia su tutto $\mathbb{R}$ (non solo fra le radici). L’equivalenza con le caratteristiche è immediata:

$$\frac{dJ}{dy} = u_0(y) - \frac{x - y}{t} = 0 \iff y + u_0(y)t = x,$$

quindi i punti stazionari di J sono le radici di $g(y) = y + u_0(y)t - x$; il minimo globale si trova confrontando i valori di J in queste radici (e sui bordi/kink quando non ce ne sono, caso che descriveremo in Sezione 6.5).

Perché non “il minimo di u_0 ”. Un’euristica diffusa consiste nel prendere, fra le radici, quella con u_0 minimo. Con un solo urto isolato il risultato coincide con quello corretto, ma appena due urti interagiscono le radici provenienti da regioni lontane (dove u_0 è basso) possono avere il valore minimo pur non essendo la caratteristica che determina la soluzione fisica. Un controesempio numerico esplicito è in Sezione 6.4 e nel test `test_soluzione_esatta.py`: con $u_0 = \sin(2\pi x)$ su $[-2, 2]$ al tempo $t = 0.5$, appena a sinistra dell’urto in $x = 0.5$ la soluzione fisica vale ≈ 0.72 , mentre la regola del minimo di u_0 restituisce ≈ -0.77 .

Quanto segue descrive l’implementazione contenuta nel codice allegato (`exact_solution` in `codice/burgers_weno.py`), passo per passo.

6.2 Caso $t \approx 0$

Se $t < 10^{-14}$ la soluzione coincide banalmente con il dato iniziale: si valuta u_0 direttamente sui punti richiesti e si esce, evitando di applicare inutilmente (e con rischio di divisione per zero) la procedura di ricerca delle radici.

6.3 Estensione del dominio di ricerca

Per trovare tutte le radici x_0 dell’equazione implicita occorre esplorare un intervallo di x_0 sufficientemente ampio: le caratteristiche possono trasportare informazione da punti anche molto lontani dal dominio $[x_{\min}, x_{\max}]$ di interesse. Si campiona u_0 su un intervallo esteso di lunghezza $3L$ (con $L = x_{\max} - x_{\min}$) per stimarne la velocità massima `maxU0`, e si pone

$$\text{ext} = \max(\text{ext_factor} \cdot L, 2 \text{maxU0} \cdot t) + 2.0,$$

dove il primo termine garantisce un’estensione minima pari a $3L$, il secondo copre la distanza massima percorribile dalle caratteristiche nel tempo t , e il margine $+2.0$ assorbe eventuali imprecisioni di campionamento.

6.4 Ricerca delle radici e selezione entropica

Definita $g(x_0) = x_0 + u_0(x_0)t - x$, si valuta g su una griglia fine di $N_s = 2000$ punti x_s nel dominio esteso, si individuano i cambi di segno tra punti adiacenti (candidati zero) e si raffina ciascuno con 64 iterazioni di **bisezione**, ottenendo una precisione ben oltre quella necessaria. Per ogni radice y trovata si valuta il funzionale entropico

$$J(y) = G(y) + \frac{(x - y)^2}{2t}$$

(la primitiva G è calcolata una volta per tutte sulla griglia estesa con la regola del trapezio e interpolata linearmente), e si tiene la radice con J minimo:

$$y^* = \arg \min\{J(y) : g(y) = 0\}, \quad u(x, t) = \frac{x - y^*}{t}.$$

L'uso di J al posto del valore $u_0(y)$ è l'unica differenza rispetto alla versione precedente della soluzione esatta: nella prima versione si prendeva la radice con u_0 minimo, che non è in generale lo stato entropico. La funzione gestisce inoltre eventuali NaN del dato iniziale sostituendoli con 0, garantendo robustezza anche per condizioni iniziali complesse come quelle usate in questo progetto.

Controesempio numerico (perché la regola del minimo di u_0 è sbagliata). Si consideri $u_0(x) = \sin(2\pi x)$ su $[-2, 2]$, che al tempo $t = 0.5$ ha un urto in $x = 0.5$ (per simmetria). Subito a sinistra dell'urto, $x = 0.486$, l'equazione $g(y) = 0$ ha più radici: quella fisica fornisce

$$u(x, t) = \frac{x - y^*}{t} \approx 0.72,$$

mentre la radice con u_0 minimo (una caratteristica partita da molto lontano) dà ≈ -0.77 . Un riferimento numerico ad alta risoluzione (schema conservativo indipendente, 2000 celle) conferma il primo valore: è il test `test_stato_entropico_accanto_all_urto` di `test_soluzione_esatta.py`.

Esempio con l'IC del progetto. Per $u_0(x) = -\sin(2x)$ su $[-\pi, \pi]$ il tempo di rottura è $T_b = \frac{1}{\max(-u'_0)} = \frac{1}{2}$: per $t = 0.4 < T_b$ la soluzione è ancora a un solo valore e le due regole coincidono (massima differenza misurata $< 10^{-6}$), mentre per $t = 1$ gli urti si sono formati e la differenza rispetto al riferimento indipendente è di due ordini di grandezza maggiore per la vecchia regola (si vedano i test e la Sezione 2.4).

6.5 Caso particolare: rarefazione di Riemann

L'algoritmo di ricerca delle radici descritto in Sezione 6.4 funziona correttamente per tutti i dati iniziali continui e per gli urti. Tuttavia, se la condizione iniziale presenta una *discontinuità verso l'alto* (rarefazione), l'equazione $x_0 + u_0(x_0)t = x$ **non ha soluzioni** nella regione del ventaglio: nessuna caratteristica arriva in quei punti. In questo caso il funzionale J non ha punti stazionari e il suo minimo cade sul *kink* della primitiva G (in corrispondenza del salto della condizione iniziale): l'implementazione descritta in Sezione 6.4 usa allora il punto di minimo di J sulla griglia e restituisce proprio il valore del ventaglio, $u = x/t$. Conviene comunque ricavare esplicitamente la soluzione del ventaglio, considerando il problema di Riemann puro:

$$u_t + uu_x = 0, \quad u(x, 0) = \begin{cases} u_l, & x < 0, \\ u_r, & x > 0, \end{cases} \quad \text{con } u_l < u_r.$$

Poiché la condizione iniziale è costante a tratti, il problema è invariante per scalatura: se moltiplichiamo x e t per una stessa costante λ , l'equazione e i dati iniziali restano identici. La soluzione deve quindi dipendere solo dal rapporto $\xi = x/t$. Poniamo $u(x, t) = \phi(\xi)$. Allora:

$$u_t = -\frac{\xi}{t}\phi', \quad u_x = \frac{1}{t}\phi',$$

e sostituendo in $u_t + uu_x = 0$:

$$-\frac{\xi}{t}\phi' + \phi\frac{1}{t}\phi' = 0 \implies \phi'(\phi - \xi) = 0.$$

Le due soluzioni elementari sono:

$$\phi' = 0 \implies \phi = \text{costante}, \quad \text{oppure} \quad \phi = \xi.$$

Per raccordare le zone piatte $\phi = u_l$ (a sinistra) e $\phi = u_r$ (a destra) senza creare un salto (condizione di entropia di Oleinik per flussi convessi), si deve usare il ramo $\phi = \xi$ nel mezzo. La soluzione debole continua è quindi:

$$u(x, t) = \begin{cases} u_l, & \frac{x}{t} \leq u_l, \\ \frac{x}{t}, & u_l \leq \frac{x}{t} \leq u_r, \\ u_r, & \frac{x}{t} \geq u_r. \end{cases} \quad (*)$$

Questa è l'onda di rarefazione centrata: un ventaglio di caratteristiche che partono tutte dall'origine $x = 0$ al tempo $t = 0$, con velocità $s \in [u_l, u_r]$. Notiamo che la formula (*) è esattamente il caso limite della formula di Lax–Oleinik: se il salto iniziale è in y_0 , la primitiva G ha il kink in y_0 e il minimo di J cade lì, dando $u = (x - y_0)/t = x/t$ (per $y_0 = 0$). Per dati iniziali generici (non costanti a tratti) il ventaglio si ottiene con la stessa formula, senza bisogno di alcun trattamento speciale: il codice allegato la calcola automaticamente, come verificato dal test `test_ventaglio_di_rarefazione`.

Sappiamo che per un punto $x(t)$ fissato abbiamo un valore u ; sappiamo che da quel punto $x(t)$ esiste una retta nel tempo per cui tutti i valori u sono uguali. Quindi dalla condizione iniziale possiamo trovare esattamente i valori della soluzione esatta per ogni t e per ogni x semplicemente espandendo tantissime rette. Per farlo vediamo che $x(t) = x_0 + u_0(x_0)t$; a questa equazione si arriva integrando $dx/dt = u$ e sfruttando il fatto che u deve essere costante lungo dx/dt , quindi $dx/dt = u_0$; poi si integra x da un tempo t_0 a un tempo t e si sposta x_0 dall'altra parte. Una volta che abbiamo quell'equazione della retta $x(t)$, possiamo trovare un qualsiasi valore di $u(x(t), t)$ solamente sapendo x_0 , ovvero il punto x nella soluzione iniziale u_0 da cui si propaga quella caratteristica, cioè la partenza della retta. Una volta trovato quel punto, basta andare a vedere che valore avevamo in $u_0(x_0)$ e copiarlo al punto $(x(t), t)$. In corrispondenza di un urto si hanno più valori di x_0 da cui partono le caratteristiche: scegliamo semplicemente il minimo di questi, e questo punto rappresenta la soluzione fisica (soluzione dell'equazione con viscosità ma con v che tende a zero). In corrispondenza di una rarefazione (problema di Riemann con discontinuità nelle condizioni iniziali) NON si ha un punto x_0 tale per cui la caratteristica arriva al punto $(x(t), t)$; allora semplicemente poniamo $u(x, t) = x/t$.

Implementazione WENO

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""Burgers 1D - schema WENO5 + SSP-RK3 e soluzione esatta entropica.

Contiene tutto quello che serve al progetto di Calcolo Scientifico
(Progetto 14) "High order FD discretization of Burgers' equation in 1D
    with
WENO":

* il solutore numerico di  $u_t + (u^2/2)_x = 0$  con
    - flux splitting di Lax-Friedrichs,
    - ricostruzione WENO5 alle interfacce di cella,
    - avanzamento temporale SSP-RK3 con dt adattato alla CFL;
* la soluzione esatta entropica, calcolata con la formula di
    **Hopf-Lax-Oleinik**:
     $u(x,t) = (f^*)'(p^*)$ ,  $p^* = (x - y^*)/t$ ,
     $y^* = \operatorname{argmin}_y [ G(y) + t f*((x-y)/t) ]$ ,  $G(y) = \int u_0$ 
    che per Burgers  $f(p) = p^2/2$  diventa
     $y^* = \operatorname{argmin}_y [ G(y) + (x-y)^2/(2t) ]$ ,  $u(x,t) = (x - y^*)/t$ .
    I punti stazionari di  $J(y)$  sono le radici dell'equazione delle
    caratteristiche  $y + u_0(y) t = x$ ; fra essi si sceglie pero' quello che
    MINIMIZZA  $J$  (condizione di entropia), non quello con  $u_0$  minimo.

Uso:
    python3 burgers_weno.py          # errori L1 + grafico su file
    python3 burgers_weno.py --show   # come sopra, ma apre la
    finestra
    python3 burgers_weno.py --nx 800 --t 0.9

La funzione `old_min_rule` (in fondo) riproduce la versione precedente -
`u = min{  $u_0(y) : y + u_0(y) t = x$  }` - e serve solo a documentare, su casi
con
piu' urti che interagiscono, che quella regola NON seleziona la soluzione
entropica (si vedano README e test_soluzione_esatta.py).
"""

import argparse
import os

import numpy as np

#
# -----
# Parametri di default (quelli del progetto)
#
# -----

AMP = 1.0          # ampiezza della condizione iniziale
XMIN, XMAX = -np.pi, np.pi
NX = 200           # celle della griglia numerica
```

```

CFL = 0.45          # coefficiente di Courant (limite pratico 0.5 per
    ↳ WENO5+RK3)
T_FINAL = 0.4       # tempo finale della simulazione
NX_EXACT = 2000     # punti della griglia di ricerca per la soluzione
    ↳ esatta
EXT_FACTOR = 3.0    # estensione minima del dominio di ricerca (in unita'
    ↳ di L)

PAD = 3             # ghost cells necessarie allo stencil di 5 punti di
    ↳ WENO5
EPS = 1e-36         # stabilizzatore dei pesi non lineari

def u0(x):
    """u(x,0) = -sin(2x): la IC del progetto (piu' urti che interagiscono
    ↳ e
    regioni di curvatura diversa)."""
    return AMP * np.sin(np.pi + 2 * x)

#
# -----
# WENO5 + SSP-RK3
#
# -----

def weno_reconstruct(f, n, side):
    """Ricostruzione WENO5 del flusso all'interfaccia j+1/2.

    side='m' usa lo stencil orientato a sinistra (f_{j-2}...f_{j+2}),
    side='p' lo stencil orientato a destra (f_{j-1}...f_{j+3});
    i pesi lineari ottimali si scambiano nei due casi.
    """
    fpad = np.pad(f, (PAD, PAD), mode='wrap')
    if side == 'm':
        v0 = fpad[PAD - 2: PAD - 2 + n] # f_{j-2}
        v1 = fpad[PAD - 1: PAD - 1 + n] # f_{j-1}
        v2 = fpad[PAD : PAD + n] # f_j
        v3 = fpad[PAD + 1: PAD + 1 + n] # f_{j+1}
        v4 = fpad[PAD + 2: PAD + 2 + n] # f_{j+2}
        p0 = (1.0 / 3.0) * v0 - (7.0 / 6.0) * v1 + (11.0 / 6.0) * v2
        p1 = -(1.0 / 6.0) * v1 + (5.0 / 6.0) * v2 + (1.0 / 3.0) * v3
        p2 = (1.0 / 3.0) * v2 + (5.0 / 6.0) * v3 - (1.0 / 6.0) * v4
        b0 = (13.0 / 12.0) * (v0 - 2 * v1 + v2) ** 2 + (1.0 / 4.0) * (v0 -
        ↳ 4 * v1 + 3 * v2) ** 2
        b1 = (13.0 / 12.0) * (v1 - 2 * v2 + v3) ** 2 + (1.0 / 4.0) * (v1 -
        ↳ v3) ** 2
        b2 = (13.0 / 12.0) * (v2 - 2 * v3 + v4) ** 2 + (1.0 / 4.0) * (3 *
        ↳ v2 - 4 * v3 + v4) ** 2
        d0, d1, d2 = 0.1, 0.6, 0.3
    else:
        # side == 'p'

```

```

v0 = fpad[PAD - 1: PAD - 1 + n] # f_{j-1}
v1 = fpad[PAD : PAD + n] # f_j
v2 = fpad[PAD + 1: PAD + 1 + n] # f_{j+1}
v3 = fpad[PAD + 2: PAD + 2 + n] # f_{j+2}
v4 = fpad[PAD + 3: PAD + 3 + n] # f_{j+3}
p0 = (1.0 / 3.0) * v0 - (7.0 / 6.0) * v1 + (11.0 / 6.0) * v2
p1 = -(1.0 / 6.0) * v1 + (5.0 / 6.0) * v2 + (1.0 / 3.0) * v3
p2 = (1.0 / 3.0) * v2 + (5.0 / 6.0) * v3 - (1.0 / 6.0) * v4
b0 = (13.0 / 12.0) * (v0 - 2 * v1 + v2) ** 2 + (1.0 / 4.0) * (v0 -
    4 * v1 + 3 * v2) ** 2
b1 = (13.0 / 12.0) * (v1 - 2 * v2 + v3) ** 2 + (1.0 / 4.0) * (v1 -
    v3) ** 2
b2 = (13.0 / 12.0) * (v2 - 2 * v3 + v4) ** 2 + (1.0 / 4.0) * (3 *
    v2 - 4 * v3 + v4) ** 2
d0, d1, d2 = 0.3, 0.6, 0.1
w0 = d0 / (b0 + EPS) ** 2
w1 = d1 / (b1 + EPS) ** 2
w2 = d2 / (b2 + EPS) ** 2
return (w0 * p0 + w1 * p1 + w2 * p2) / (w0 + w1 + w2)

def compute_L(u, dx):
    """Residuo spaziale L(u) = -(F_{j+1/2} - F_{j-1/2})/dx (forma
    conservativa)."""
    n = len(u)
    f = 0.5 * u * u # flusso fisico f(u) = u^2/2
    alpha = np.max(np.abs(u)) # velocita' caratteristica
    # massima
    fp = 0.5 * (f + alpha * u) # splitting di Lax-Friedrichs
    fm = 0.5 * (f - alpha * u)
    F_hat = weno_reconstruct(fp, n, 'm') + weno_reconstruct(fm, n, 'p')
    F_ext = np.concatenate([F_hat[-1]], F_hat) # condizioni periodiche
    return -(F_ext[1:] - F_ext[:-1]) / dx

def ssp_rk3(u, dx, dt):
    """Terzo ordine SSP (Shu-Osher) in tre stadi."""
    L0 = compute_L(u, dx)
    u1 = u + dt * L0
    L1 = compute_L(u1, dx)
    u2 = 0.75 * u + 0.25 * (u1 + dt * L1)
    L2 = compute_L(u2, dx)
    return (1.0 / 3.0) * u + (2.0 / 3.0) * (u2 + dt * L2)

def solve_weno(u0_arr, dx, T_final, cfl):
    """Integra in tempo con dt adattato alla condizione CFL."""
    u = u0_arr.copy()
    t = 0.0
    while t < T_final:

```

```

        maxU = np.max(np.abs(u))
        dt = cfl * dx / maxU if maxU > 1e-14 else 1e-12
        if t + dt > T_final:
            dt = T_final - t
        u = ssp_rk3(u, dx, dt)
        t += dt
    return u

#
# -----
# SOLUZIONE ESATTA - formula di Hopf-Lax-Oleinik
#
# -----
def primitiva(u0_func, ys, u0sy=None):
    """G(y) = int_{ys[0]}^{y} u0(s) ds, con la regola del trapezio."""
    if u0sy is None:
        u0sy = np.nan_to_num(np.array([u0_func(y) for y in ys]), nan=0.0)
    G = np.zeros(np.size(ys))
    G[1:] = np.cumsum(0.5 * (u0sy[1:] + u0sy[:-1]) * np.diff(ys))
    return G

def exact_solution(u0_func, x_vec, t, Ns=NX_EXACT, ext_factor=EXT_FACTOR):
    """Soluzione entropica di Burgers (formula di Hopf-Lax-Oleinik).

    Per  $f(u) = u^2/2$  si ha  $f^*(p) = p^2/2$ , quindi

    
$$u(x,t) = (x - y^*)/t, \quad y^* = \operatorname{argmin}_y [ G(y) + (x-y)^2 / (2t) ].$$


    I punti stazionari del funzionale  $J(y) = G(y) + (x-y)^2/(2t)$  sono le radici di  $g(y) = y + u0(y) t - x$  (le caratteristiche che arrivano in  $x$ );
    fra i candidati si sceglie quello che MINIMIZZA J - questa e' la condizione di entropia, e non equivale a prendere il minimo di u0.
    Se nessuna caratteristica arriva in x (ventaglio di rarefazione) il minimo
    di J cade sul bordo/kink della primitiva e la stessa formula restituisce
    u = x/t, cioe' la nota soluzione del ventaglio.
    """
    x_vec = np.asarray(x_vec, dtype=float)
    if t < 1e-14: # t = 0: la IC stessa
        return np.array([u0_func(xi) for xi in x_vec])

    xmin, xmax = x_vec[0], x_vec[-1]
    L = xmax - xmin
    # max|u0| = max|f'(u0)| sul dominio esteso -> dimensione
    # dell'estensione
    maxU0 = 0.1

```

```

Nc = 500
for j in range(Nc + 1):
    v = u0_func(xmin - L + j * (3.0 * L / Nc))
    if np.isfinite(v) and abs(v) > maxU0:
        maxU0 = abs(v)
ext = max(ext_factor * L, 2.0 * maxU0 * t) + 2.0
xs = np.linspace(xmin - ext, xmax + ext, Ns)
u0s = np.nan_to_num(np.array([u0_func(y) for y in xs]), nan=0.0)
G = primitiva(u0_func, xs, u0s)

def G_at(y):
    i = np.clip(np.searchsorted(xs, y), 1, Ns - 1)
    frac = (y - xs[i - 1]) / (xs[i] - xs[i - 1])
    return G[i - 1] + frac * (G[i] - G[i - 1])

def J(y, x):
    return G_at(y) + (x - y) ** 2 / (2.0 * t)

out = np.zeros(x_vec.size)
for idx, x in enumerate(x_vec):
    gs = xs + u0s * t - x
    # (1) candidati: zeri di g(y) = 0 (punti stazionari di J,
    #     bisezione)
    best_y, best_J = None, np.inf
    for j in np.where(gs[:-1] * gs[1:] <= 0)[0]:
        lo, hi, glo = xs[j], xs[j + 1], gs[j]
        for _ in range(64):
            mid = 0.5 * (lo + hi)
            gm = mid + u0_func(mid) * t - x
            if glo * gm <= 0:
                hi = mid
            else:
                lo, glo = mid, gm
        y = 0.5 * (lo + hi)
        Jv = J(y, x)
        if Jv < best_J:
            best_y, best_J = y, Jv
    # (2) fallback: se nessuna caratteristica arriva in x (vento di
    #     rarefazione) si minimizza J sulla griglia, con raffinamento
    if best_y is None:
        j0 = int(np.argmin(G + (x - xs) ** 2 / (2.0 * t)))
        a, b = xs[max(0, j0 - 1)], xs[min(Ns - 1, j0 + 1)]
        phi = (np.sqrt(5.0) - 1.0) / 2.0
        c, d = b - phi * (b - a), a + phi * (b - a)
        fc, fd = J(c, x), J(d, x)
        for _ in range(80):
            if fc < fd:
                b, d, fd = d, c, fc
                c = b - phi * (b - a)
                fc = J(c, x)

```

```

        else:
            a, c, fc = c, d, fd
            d = a + phi * (b - a)
            fd = J(d, x)
            best_y = 0.5 * (a + b)
            out[idx] = (x - best_y) / t # u(x,t) = (x - y*)/t
    return out

def old_min_rule(u0_func, x_vec, t, Ns=NX_EXACT, ext_factor=EXT_FACTOR):
    """Versione PRECEDENTE (non corretta) della soluzione esatta:

        
$$u(x,t) = \min\{ u_0(y) : y + u_0(y) t = x \}.$$


    Serve solo a documentare l'errore corretto in questo progetto: quando
    → piu'
    urti interagiscono, il minimo di u0 fra le caratteristiche che
    → arrivano in
    (x,t) non e' la selezione entropica (si veda
    → test_soluzione_esatta.py).
    """
    x_vec = np.asarray(x_vec, dtype=float)
    if t < 1e-14:
        return np.array([u0_func(xi) for xi in x_vec])
    xmin, xmax = x_vec[0], x_vec[-1]
    L = xmax - xmin
    maxU0 = 0.1
    Nc = 500
    for j in range(Nc + 1):
        v = u0_func(xmin - L + j * (3.0 * L / Nc))
        if np.isfinite(v) and abs(v) > maxU0:
            maxU0 = abs(v)
    ext = max(ext_factor * L, 2.0 * maxU0 * t) + 2.0
    xs = np.linspace(xmin - ext, xmax + ext, Ns)
    u0s = np.nan_to_num(np.array([u0_func(y) for y in xs]), nan=0.0)
    out = np.zeros(x_vec.size)
    for idx, x in enumerate(x_vec):
        gs = xs + u0s * t - x
        cands = []
        for j in np.where(gs[:-1] * gs[1:] <= 0)[0]:
            lo, hi, glo = xs[j], xs[j + 1], gs[j]
            for _ in range(64):
                mid = 0.5 * (lo + hi)
                gm = mid + u0_func(mid) * t - x
                if glo * gm <= 0:
                    hi = mid
            else:
                lo, glo = mid, gm
            u0c = u0_func(0.5 * (lo + hi))
            if np.isfinite(u0c):

```

```

        cands.append(u0c)
    out[idx] = min(cands) if cands else
    u0s[int(np.argmin(np.abs(gs)))]
return out

#
-----
# Main
#
-----
def main():
    ap = argparse.ArgumentParser(
        description="Burgers 1D: WENO5 + SSP-RK3 contro la soluzione
        esatta di Hopf-Lax-Oleinik")
    ap.add_argument("--nx", type=int, default=NX, help="celle della
        griglia numerica")
    ap.add_argument("--cfl", type=float, default=CFL, help="coefficiente
        CFL")
    ap.add_argument("--t", type=float, default=T_FINAL, help="tempo
        finale")
    ap.add_argument("--nx-exact", type=int, default=NX_EXACT,
        help="punti della griglia di ricerca della soluzione
        esatta")
    ap.add_argument("--show", action="store_true", help="apre la finestra
        del grafico")
    ap.add_argument("--no-figure", action="store_true", help="non salva il
        grafico")
    args = ap.parse_args()

    import matplotlib
    if not args.show:
        matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    x, dx = np.linspace(XMIN, XMAX, args.nx, endpoint=False, retstep=True)
    x = x + 0.5 * dx
    u_ini = u0(x)
    u_exact = exact_solution(u0, x, args.t, args.nx_exact, EXT_FACTOR)
    u_weno = solve_weno(u_ini, dx, args.t, args.cfl)
    diff = np.abs(u_weno - u_exact)
    err_medio = float(diff.mean())
    err_l1_rel = float(diff.sum() / np.abs(u_exact).sum())

    print(f"Nx={args.nx} CFL={args.cfl} T={args.t}")
    print(f"errore medio |u_num - u_esatta| = {err_medio:.6e}")
    print(f"errore L1 relativo (vs ||u_esatta||_1) = {err_l1_rel:.6e}")

    plt.figure(figsize=(10, 6))
    plt.plot(x, u_ini, 'k--', linewidth=1.5, label='t=0')

```

```

plt.plot(x, u_exact, 'b-', linewidth=2.5, label='Esatta
(Hopf-Lax-Oleinik)')
plt.plot(x, u_weno, 'r-', linewidth=2, label='WENO5')
plt.xlabel('x')
plt.ylabel('u(x,t)')
plt.title(f'Burgers, t={args.t:.3f}, Nx={args.nx}, CFL={args.cfl}')
plt.legend()
plt.grid(alpha=0.3)
if not args.no_figure:
    out =
    os.path.normpath(os.path.join(os.path.dirname(os.path.abspath(__file__)),
                                os.pardir, "figure"))

    os.makedirs(out, exist_ok=True)
    path = os.path.join(out, "burgers_weno.png")
    plt.savefig(path, dpi=130, bbox_inches="tight")
    print(f"grafico salvato in {path}")
if args.show:
    plt.show()

if __name__ == "__main__":
    main()

```